void SSABuilder::reset() -> loop over mVarDefs and delete each submap value. Then loop over mIncompletPhis and delete each subphi value. Clear both mVarDefs and mIncompletePhis. Then clear mSealedBlocks. This resets the state.

void SSABuilder::addBlock(BasicBlock* block, bool isSealed /* = false */) -> create a new submap and insert it into mVarDefs. Then create a new subphi and insert into mIncompletePhis. If the isSealed flag is true then call sealBlock() on the current basic block to seal it. It is basically the opposite of reset() function

void SSABuilder::writeVariable(Identifier* var, BasicBlock* block, Value* value) -> look up the block parameter in mVarDefs and assign it to a variable pointer. Then dereference that pointer and assign the parameter value to the index var.

Value* SSABuilder::readVariable(Identifier* var, BasicBlock* block) -> get the submap for the block in the same way as writeVariable(). Check if the identifier var is defined inside of this block by searching through the submap. If it is, then return that value. Otherwise return the value of readVariableRecrusive(var, block).

Value* SSABuilder::readVariableRecursive(Identifier* var, BasicBlock* block) -> first get the phi type using var->llvmType(block->getContext()) since the address of the variable is null, and get the insert location with getFirstNonPHIIt(). Then get the predecessor of the block. If the block is not sealed, create an empty phi and add it to the incomplete phis and return the value. Otherwise, if the block has a single predecessor, return readVariable() on the predecessor, no phi needed. Otherwise create a phi, write it to the block first so the function doesnt recurse forever, and then call addPhiOperands(). At the end call writeVariable() with the result and return it.

Value* SSABuilder::addPhiOperands(Identifier* var, PHINode* phi) -> get the parent block of the phi. Loop over all the predecessors of that block and for each one call addIncoming() on the phi with the value from readVariable(var, predecessor) and the predecessor block. Then return tryRemoveTrivialPhi(phi).

Value* SSABuilder::tryRemoveTrivialPhi(PHINode* phi) -> loop over the operands of the phi. If the operand is the same as the one already found or the phi itself then skip it. If a different value was already found, the phi merges two values so it is not trivial and

return the phi. Otherwise save the operand as same. If same is still null after the loop, set it to an undef value. Then save all the users of the phi that are other phis into a vector (no duplicates). Replace all uses of the phi with same, and update mVarDefs so any entry that pointed to the phi now points to same. Erase the phi and then call tryRemoveTrivialPhi() on each saved user since they might be trivial now. Return same.

void SSABuilder::sealBlock(BasicBlock* block) -> get the subphi for the block from mIncompletePhis. Loop over it and call addPhiOperands() on each variable and phi pair to finish the incomplete phis. Then clear the subphi and add the block to mSealedBlocks.

AST_EMIT(ASTIfStmt) -> add the end block unsealed after creating it. If there is an else, add the else block sealed right after the conditional branch. After the conditional branch add the then block sealed. After the then body branches to end, seal the end block since it has all its predecessors.

AST_EMIT(ASTWhileStmt) -> add the cond block unsealed before emitting the condition. After the conditional branch, add the body block sealed and the end block unsealed. Only add the back edge branch if the body doesnt already have a terminator. Then seal the cond block and the end block.

Note: for part 2 I created the helper functions as suggested in algorithm 5 of the paper attached

bool RedundantPhiRemoval::runOnFunction(Function &F) -> set changeFlag to false. Loop over every block in the function and every phi in the block and add them to a set. Call removeRedundantPhis() on the set and return changeFlag.

void RedundantPhiRemoval::removeRedundantPhis(const std::set<PHINode*>& phis) -> make a graph and add a node for each phi. For each operand of a phi, if it is a phi that is also in the set, add an edge from the phi to the operand. Then call getSCCs() on the graph, which already gives them in reverse topological order, and call processSCC() on each one.

void RedundantPhiRemoval::processSCC(const std::set<Value*>& strongConnections) -> make two sets, inner and outerOps. For each phi in the scc, loop over its operands and if an operand is not in the strong connetions set add it to outerOps and mark the phi as not inner. If all operands of the phi are in the strong connections set then add it to inner. After the loop, if outerOps has exactly one value, call replaceSCCByValue() with that value. If it has more than one, call removeRedundantPhis() on inner. Unlike the paper this does not skip scc of size 1 so trivial phis are removed too.

void RedundantPhiRemoval::replaceSCCByValue(const std::set<Value*>& strongConnections, Value* toReplaceWith) -> loop over the strong connections set and replace all uses of each phi with toReplaceWith. Then loop again and erase each phi. Then set changeFlag to true.